

CS396 Project Phase 1 Report

epaData: A Data Management System for EPA Power-Sector Emissions Data

Raaj Banga

Arnav Bawankule

October 2026

Abstract

epaData is a web application for managing the annual operating and emissions data of U.S. power-plant units published through the EPA Clean Air Markets Program Data (CAMPD) service. It retrieves data from the CAMPD API, validates and imports CSV and Excel files, stores the result in a relational SQLite database, and supports searching, ranking, comparing, inspecting, and exporting it. The committed database holds 1,619 facilities, 5,204 generating units, and 50,947 unit-year records for the reporting years 2015–2026. Imports made through the interface are compared with the stored data and saved only on the user’s approval; rows that fail validation are retained in a data-quality report, and every record can be traced to the retrieval or file that last wrote it. Searches may be expressed as filters, numeric ranges, or plain-English descriptions, which a rule-based parser translates into the same filters. This report documents the design, implementation, and evaluation of the system and the limitations carried into Phase 2.

Contents

1	Project description	4
1.1	Problem	4
1.2	Intended users	4
1.3	Objectives	4
1.4	Scope	5
2	Functions implemented	5
2.1	Home page	5
2.2	Data retrieval from the EPA API	6
2.3	File upload and validation	6
2.4	Search	6
2.5	Facility and unit details	7
2.6	Comparison, map, and audits	7
2.7	Download	8
2.8	Requirement traceability	8
2.9	Planned functions not completed or changed	9

3	System design	9
3.1	Architecture	9
3.2	Life cycle of a search	10
3.3	Database schema	11
3.4	Data flow and provenance	12
4	Technical details	13
4.1	Languages	13
4.2	Database system	13
4.3	Frameworks and libraries	13
4.4	External services	14
4.5	Scripts and configuration	14
5	Highlighted features	15
5.1	Comparing the database with the EPA API before saving	15
5.2	Upload validation and the data-quality report	17
5.3	Description-based search	18
5.4	Multi-constraint search and ranking in SQL	21
5.5	Physical-sanity audits	22
5.6	Provenance and export	23
6	Testing and results	24
6.1	Approach	24
6.2	Unit tests	24
6.3	Representative test cases	25
6.4	Performance	25
6.5	Known issues	26
6.6	Assessment against the objectives	26
7	Discussion	27
7.1	Effectiveness of techniques and tools	27
7.2	Technical choices and alternatives	28
7.3	Challenges and lessons learned	29
8	Limitations and future improvements	30
8.1	Current limitations	30
8.2	Future improvements	31
9	Individual contributions	31
10	References and acknowledgments	32

10.1 Data	32
10.2 Reused code and assets	32
10.3 Generative AI disclosure	32

1 Project description

1.1 Problem

Fossil-fuel power plants subject to federal programs such as the Acid Rain Program and the Cross-State Air Pollution Rule must report their fuel use, electricity generation, and emissions of sulfur dioxide (SO₂), nitrogen oxides (NO_x), and carbon dioxide (CO₂). The U.S. Environmental Protection Agency publishes these reports through CAMPD, both as downloadable files and through a REST interface, the CAM API. For every generating unit the data give annual operating time, gross load, heat input, and emitted mass, together with the unit's fuel, type, and pollution-control equipment.

The data are authoritative but difficult to query. Answering a question such as “*Which coal units in Kentucky emitted more than 500,000 tons of CO₂ in 2025?*” requires calling the correct endpoint with the correct parameters, joining the result with facility metadata, and applying a numeric threshold; a comparison across years requires one request per year. Analysts who receive CAMPD extracts as spreadsheets have no simple means of checking them for missing values, out-of-range numbers, or repeated facility-unit-year records, or of seeing how they differ from data already held. Once several sources are combined, the origin of each figure is easily lost.

1.2 Intended users

The system serves three groups: students and researchers who need emissions data without writing API code; environmental and energy analysts who combine CAMPD data with their own spreadsheet extracts and need those files validated and reconciled; and reviewers and regulators who must trace a figure to its source and see which records were rejected and why. No user is assumed to know SQL or the structure of the CAM API, so searches are expressed through forms, tables, and plain-English descriptions.

1.3 Objectives

The course project description defines twelve learning objectives for the two-phase project. Phase 1 addresses the nine concerned with data management:

- O1. Design a web-based information system with a front end and a database back end.
- O2. Develop server routes, page templates, forms, and interactive web components.
- O3. Design and implement a relational database using SQLite and an object-relational mapper.
- O4. Retrieve environmental data from public sources or import downloaded files.
- O5. Validate, clean, transform, and store uploaded data.
- O6. Implement database search, filtering, sorting, and pagination.
- O7. Export selected data and calculated results.
- O11. Test and deploy a complete data-driven web application.
- O12. Document data sources, assumptions, equations, units, and system limitations.

Objectives O8–O10 (environmental indicators, multi-indicator scoring, and interactive charts of evaluation results) belong to Phase 2. Section 6 assesses each Phase 1 objective.

1.4 Scope

Phase 1 delivers a data-management system rather than an evaluation tool. It covers retrieval of annual apportioned emissions from the CAM API; upload of CSV and Excel files in the CAMPD column layout; validation and storage in SQLite; basic, multi-constraint, ranking, and description-based search; facility and unit details and comparison; and CSV export of datasets, search results, selections, valid and invalid records, and provenance. The EPA TRACI characterisation factors [1], environmental-impact indicators, and the weighted scoring model are deferred to Phase 2. Sub-annual (hourly to monthly) data are displayed on demand from the API but not stored, and the application runs locally (Section 8).

2 Functions implemented

epaData is a web application with six pages, reached from a navigation bar that orders them by workflow: *Home*, *Explore*, and *Map* for looking at the stored data, then *Retrieve*, *Upload*, and *Download* for moving data in and out. Facility, unit, and comparison details open in a single near-full-screen dialog over whichever page is active; the open facility or unit is recorded in the URL (`?facility=3` or `?unit=...`), so a detail view can be linked to and survives a reload. This section describes each function from the user’s perspective; Section 3 explains how the functions are built, and Section 5 examines the most significant of them in detail.

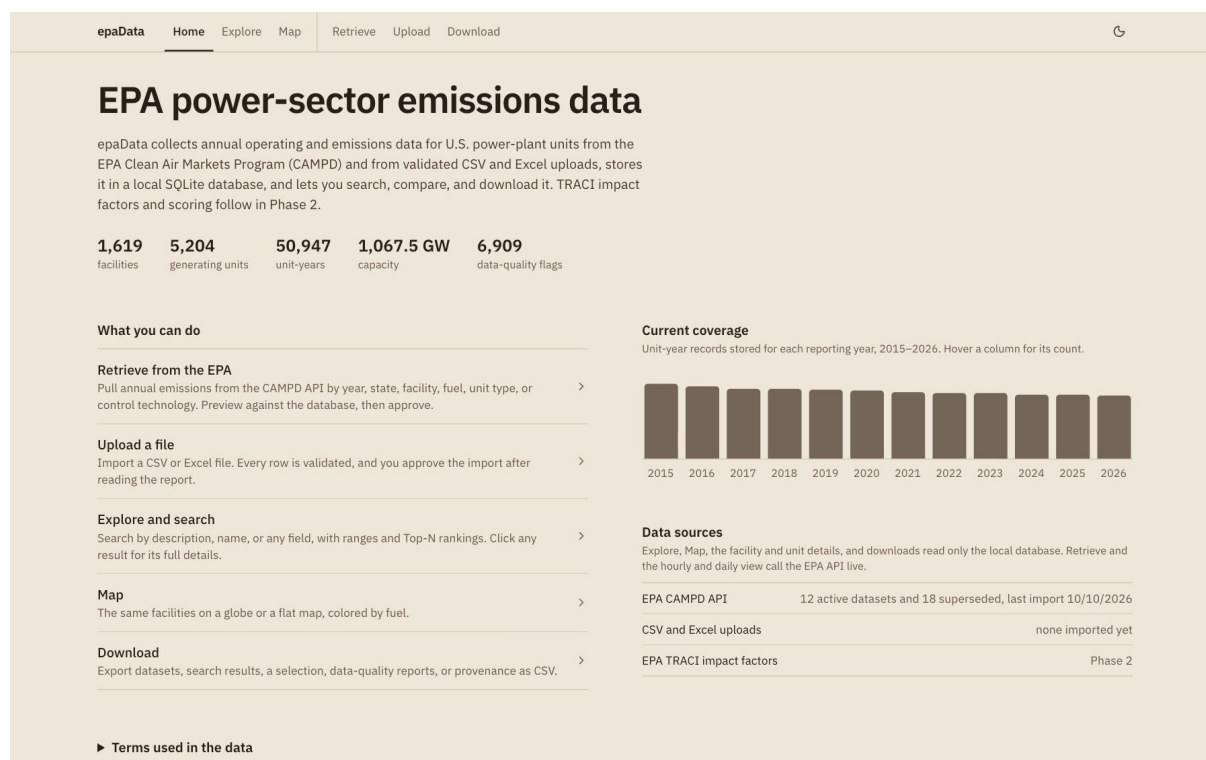


Figure 1: Home page: introduction and summary figures, links to every function in workflow order, unit-year coverage per reporting year, and the data sources.

2.1 Home page

The home page (Figure 1) introduces the project and summarises the database in five figures: facilities, generating units, unit-years, installed capacity, and data-quality flags. A list links to every function in the order in which data moves through the application. A column chart gives

the number of unit-year records stored for each reporting year, and a data-sources list reports, for each source, its active and superseded datasets and the date of its last import (for example, “EPA CAMPD API: 12 active datasets and 18 superseded”), together with a statement of which views read the local database and which call the EPA API live. A collapsible glossary defines the domain terms used in the tables.

2.2 Data retrieval from the EPA API

The **Retrieve** page names its retrieval method, the CAM API’s apportioned annual emissions endpoint, and queries it with the filters the specification requires: a range of reporting years, state, facility IDs, fuel type, unit type, and control technology. The fuel, unit-type, and control options are loaded from CAMPD’s master-data endpoints, so every choice is a value the API accepts, and a row of sample retrievals fills the form in one click. While the form is edited, a section titled *Already in the local database* reports how many matching unit-years are stored and which dataset last wrote them. **Preview** fetches the data without saving and classifies every record as new, changed, unchanged, or dropped, showing the stored and incoming value of each differing field; **Approve and save** stores each year as a new dataset. A history table lists every past retrieval with its parameters, date, status, and counts. The API key is held in a server-side environment variable and never reaches the browser. Section 5.1 presents a worked example.

2.3 File upload and validation

The **Upload** page accepts a CSV or Excel file of up to 100 MB by drag-and-drop or file picker; the extension and size are checked in the browser and again on the server. A Python script maps the file’s headers to database columns, validates every row, and detects repeated facility-unit-year records. Before anything is written, the page reports the counts of valid, rejected, duplicate, and flagged rows; compares the valid records with the database; and lists, on separate tabs, each rejected row with its column, value, and reason, the repeats within the file, the records already in the database with the fields that would change, and the column mapping. A daily or hourly CAMPD file, whose metrics have no place in an annual schema, is recognised as such, and the page states that only its facility and unit attributes will be stored. On approval, the valid records, the rejected and duplicate rows, the upload metadata, and a copy of the original file are stored, and the rejected rows can be downloaded directly from the confirmation. Below the drop zone, a history table lists every past upload with the same counts as the retrieval history (Section 5.2).

2.4 Search

The **Explore** page has three tabs that share one search form: *Facilities*, with one row per plant and its totals; *Unit-years*, with one row per unit and reporting year, the grain of the stored records; and *Audit flags*, described below.

Basic search. A single search box accepts either a description or a name. Below it, the basic fields of the specification are always visible: state, reporting year, primary fuel, unit type, facility ID, and unit ID. An *Advanced* panel adds county, NERC region, secondary fuel, SO₂/NO_x/PM control, operating status, audit flag and severity, and record origin (API or file upload); it opens by itself whenever one of its filters is set, so no active filter is ever hidden.

Multi-constraint and range search. All filters combine conjunctively. Inclusive minimum and maximum bounds are available for reporting year, operating time, gross load, heat input,

and CO₂, SO₂, and NO_x mass, and every active filter appears as a removable chip above the results (Section 5.4).

Description-based search. A sentence typed into the search box, such as “coal units in Kentucky with high CO₂”, is translated on submission into the same filters, which appear as chips labelled *Interpreted as* and in the filter controls for further refinement; words that could not be interpreted are listed rather than ignored. Text in which nothing is recognised, such as a plant name, falls back to a name search, so the one box serves both purposes (Section 5.3).

Ranking and comparison. Any metric column can be sorted, and a *Ranking* control in the Advanced panel retains the first N rows of that order, overall or per state, which covers the specification’s top- N , bottom- N , and group-and-rank searches. Ticking two to four facilities or units opens a dock at the bottom of the window from which they are compared side by side. Historical searches use the year range or the unit details, which list every reporting year of a unit.

Result handling. Results are paginated (10 to 100 rows per page) and sortable on every column with a meaningful order, and clicking a row opens its details. A **Download CSV** button exports every matching row across all pages, and *More options* carries the search to the Download page. The tab, filters, sort, and page are encoded in the URL query string, so any search can be bookmarked, shared, and reloaded.

2.5 Facility and unit details

The facility details show the plant’s identification (ID, county, state, coordinates, EPA and NERC region, owner, operating role, fuels), its latest year of operation and emissions with a short generated summary, and four tabs: the year-by-year records, each expandable to its units and the dataset that supplied them; the units with their fuels and controls; a granular view of hourly, daily, weekly, or monthly data fetched live from the API, alongside yearly totals from the database; and the audit flags. The unit details (Figure 12) show identification, fuel and controls, programs, every reporting year with the dataset that supplied it and the controls and programs reported for that year, and any sanity flags.

2.6 Comparison, map, and audits

The comparison (Figure 2) places two to four facilities or units side by side for the latest reporting year: capacity, generation, operating hours, capacity factor, carbon intensity, heat rate, fuels, and control coverage. The **Map** page draws every facility with coordinates on a three-dimensional globe or a two-dimensional Leaflet map, sized by capacity or CO₂ and coloured by fuel, with the Explore filters carried over in the URL (Figure 3); the fuel legend doubles as a filter, and the 37 facilities for which CAMPD has no coordinates are listed beneath the map rather than silently omitted. The Audit flags tab of the Explore page lists every physical-sanity flag with its rule, severity, the values that tripped it, and the dataset of the flagged record, paginated, sortable, and filtered by the same search form.

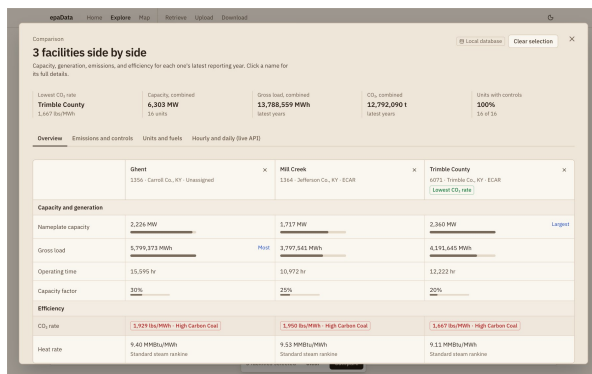


Figure 2: Comparison of three Kentucky plants.

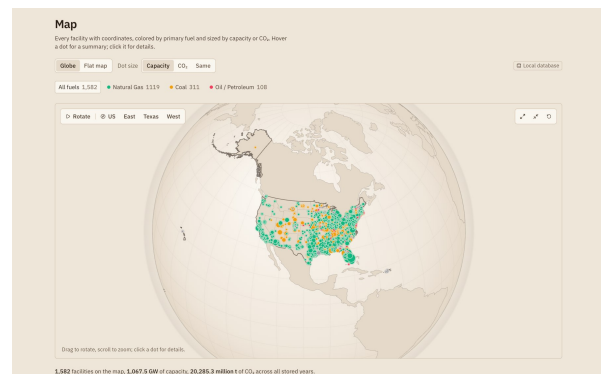


Figure 3: Map page: globe view, coloured by fuel.

2.7 Download

The **Download** page (Figure 13) provides the six CSV exports the specification requires, chosen in three numbered steps (what to download, which dataset or filters, and the file format): a complete dataset; its valid records; its invalid-record report (rejected and duplicate rows with their original columns, and records flagged by a sanity rule); the current search results; the comparison selection; and the provenance table (Section 5.6).

2.8 Requirement traceability

Table 1 maps each requirement of the course project description, and the rubric's description-search item, to the function that satisfies it.

Table 1: Requirements and where epaData meets them.

Requirement	Implementation	Status
§5 Retrieval via CAM API, key on server	Retrieve page; <code>CAMPD_API</code> read only on the server	Met
§5 Filters: year, state, facility, fuel, unit type, control	Retrieve parameters	Met
§5 Preserve source, date, parameters, counts, year	<code>datasets</code> row per year; history table	Met
§6 Steps 1–11 of the upload process	Upload page, <code>parse_import.py</code> , approval step, archive	Met
§6 Never discard invalid records silently	<code>import_issues</code> ; invalid-record export	Met
§7 Dataset, facility, unit, annual-record tables	Six tables, keys and cascades; controls and programs per annual record (Section 3)	Met
§8.1 Basic search fields	Search form; <i>Advanced</i> panel	Met
§8.2 Multi-criteria and range search	AND-combined filters, seven min/max ranges	Met
§8.3 Top- <i>N</i> , bottom- <i>N</i> , group-and-rank, comparison, history	Ranking control, comparison, unit details, year range	Met
§8.4 Readable, paginated, sortable, linked, downloadable, reproducible	Tables, pagination, detail dialog, CSV, URL state	Met
§8.4 Indexes on searched fields	Fourteen secondary and unique indexes	Met
§9 Six pages	Home, Explore, Map, Retrieve, Upload, and Download pages; detail dialog	Met
§10 Six download types, CSV	Download page, <code>/api/export</code>	Met

Requirement	Implementation	Status
Rubric 6: description-based search	Rule-based parser, optional language-model fallback	Met

2.9 Planned functions not completed or changed

Table 2 compares the Phase 1 proposal with the delivered system and gives the reason for each difference.

Table 2: Proposal versus delivered system.

Proposed	Delivered	Reason
Prisma ORM	Drizzle ORM	Drizzle’s SQL-like query builder expresses the window functions and correlated subqueries the search needs (Section 7).
Zod and Fast-CSV for upload parsing	Python script (<code>csv</code> , <code>openpyxl</code>)	The specification requires the file to be read with Python; Excel support came with it.
CSV upload only	CSV and Excel	Required by the specification.
Leaflet map with coordinate clustering	Leaflet map with scaled markers, and a D3 globe	Not completed: markers are sized and coloured instead of clustered.
Anomaly detection including spikes	Four physical-sanity rules	Not completed: no year-over-year spike detection.
Large-dataset stress tests	Timing measurements on the full database (Section 6)	Not completed as a separate test suite.
Hosted deployment	Runs locally	Dropped; SQLite on serverless hosting needed workarounds (Section 8).
—	DB-vs-API preview, provenance, origin filter, description search, granular time series, globe	Added beyond the proposal.

3 System design

3.1 Architecture

epaData is a single Next.js application containing both the user interface and the server (Figure 4). The first page load is rendered on the server with its data already fetched; subsequent interactions call the server through typed tRPC procedures or two HTTP route handlers. Every database query is built with the Drizzle ORM and executed against a single SQLite file through the libSQL client. Only two kinds of work leave the Node.js process: reading uploaded files, which is delegated to a Python subprocess, and calls to the EPA CAM API and, optionally, the OpenRouter language-model service.

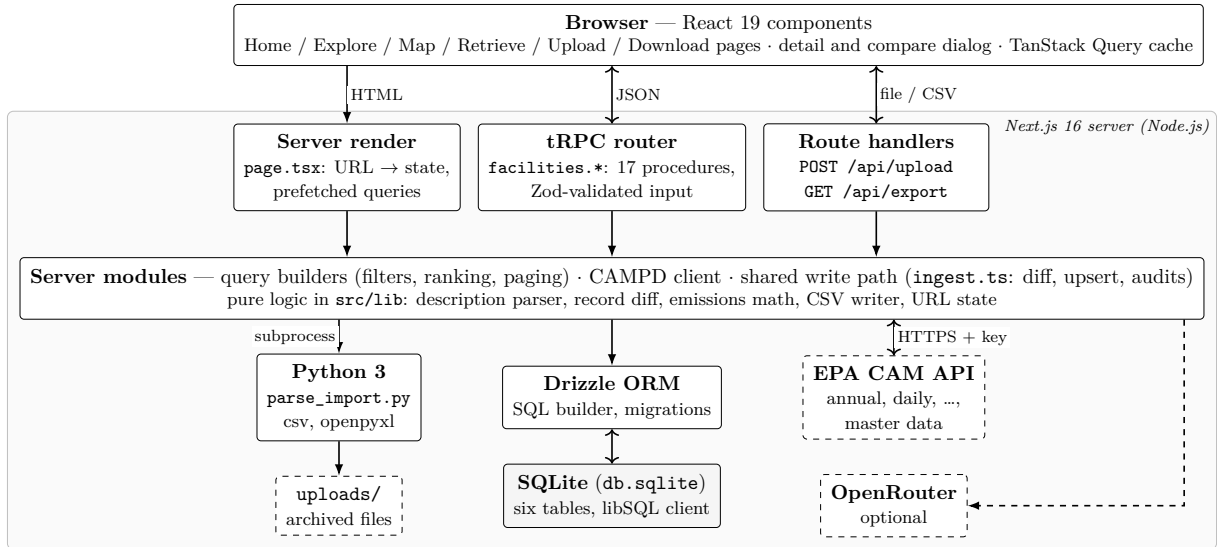


Figure 4: Architecture. Solid boxes run inside the Next.js server process; dashed boxes are external services or files. The browser never contacts the EPA API or OpenRouter directly, so their keys remain on the server.

The source tree mirrors these layers. `src/app` contains the page and its React components. `src/server/api` defines the tRPC router, whose procedures validate their input with Zod and delegate to `src/server`, which holds the query builders, the CAMPD client, the upload and export modules, and the shared write path. `src/lib` contains logic with no database or network access: filter and URL state, the description parser, record comparison, emissions arithmetic, and CSV formatting. Keeping this logic pure makes it testable with plain unit tests (Section 6). The command-line scripts in `scripts/` import the same modules, so a command-line sync and an in-app retrieval execute identical code.

3.2 Life cycle of a search

Tracing one search through the system illustrates how the components interact. Suppose a user opens the Unit-years tab of the Explore page, selects Kentucky and Coal, sets the year to 2025, and enters a minimum CO₂ of 500,000 tons.

1. Each change updates a filter object in React state, which the pure function `explorerSearchParams` mirrors into the URL, so the browser history and any shared link carry the full search:
`/explore?tab=units&stateCode=KY&primaryFuel=Coal&year=2025&co2MassTonsMin=500000`
2. A deferred copy of the filters is passed to `api.facilities.getUnitYears.useQuery`. Deferral keeps typing responsive; TanStack Query keeps the previous page on screen until the new one arrives and caches each distinct query.
3. On the server, Zod validates the input against `facilityFilterSchema` extended with paging and sort fields, rejecting unknown sort keys and out-of-range page sizes.
4. `rankedUnitYears` builds a single SQL statement: `annual_records` joined to `units`, `facilities`, and `datasets`; a WHERE clause assembled from facility, unit, and record condition builders; and a `ROW_NUMBER()` window over the requested sort (Listing 4).
5. `pageOf` runs two queries over the same ranked subquery, one for the page of rows and one for the total count. SQLite answers in about 6 ms using the `(year, co2_mass_tons)` index.

- The table renders 25 rows across three pages. Its **Download CSV** button calls `/api/export?type=search&...` with the same query string, which the handler parses with the same `parseExplorerParams` function before running `rankedUnitYears` without a page limit, so the file contains exactly the rows the table showed.

On reload, `explore/page.tsx` parses the URL on the server, prefetches the same query, and sends the rendered table with its data embedded, so a shared link opens directly on the results.

3.3 Database schema

Figure 5 shows the six tables and their relationships. Following the relational model [4], each table holds one kind of entity identified by a key, and relationships are expressed through foreign keys. The design follows the real-world hierarchy of the data: a *facility* (a power plant, identified by its EPA ORISPL code) houses one or more *units* (boilers, turbines, or engines), each of which reports one *annual record* per year. Two tables record provenance: *datasets* has one row per retrieval or upload, and *import_issues* one row per rejected or skipped source row. The sixth table, *data_audit_logs*, holds the physical-sanity flags raised on annual records.

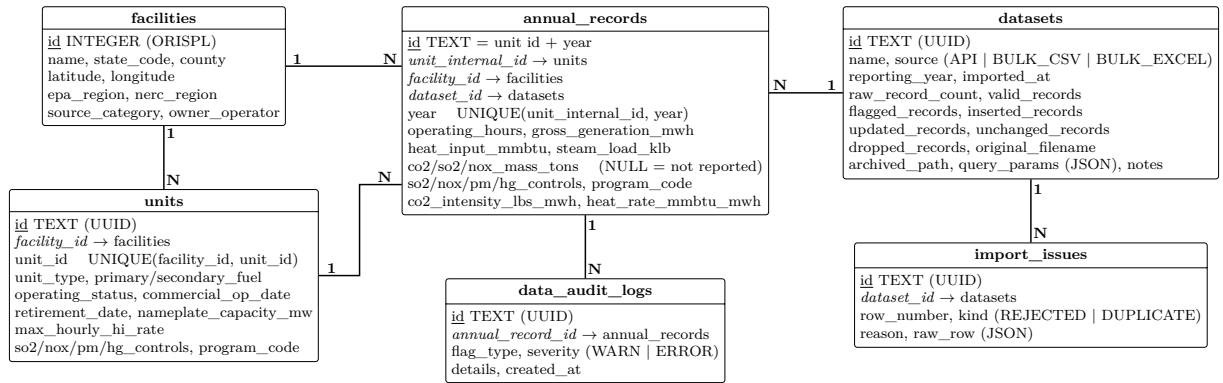


Figure 5: Entity-relationship diagram. Underlined columns are primary keys; italic columns are foreign keys (arrows name the referenced table). All foreign keys cascade on delete.

Keys and integrity rules. Integrity rules are enforced by the database rather than by application code alone. `facilities.id` is the EPA facility ID itself: being stable and meaningful, it serves directly as the primary key and appears in URLs and exports. EPA’s unit ID (“1”, “CT1”, ...) is unique only within a facility, so `units` has a surrogate UUID key and a unique index on (`facility_id`, `unit_id`), which upserts use as their conflict target. An annual record is unique per (`unit_internal_id`, `year`), the specification’s “facility-unit-year”, and its primary key is derived from the same pair (`{unitInternalId}_{year}`), so re-importing a year updates the existing row rather than inserting a duplicate. Metric columns are nullable `REAL`: `NULL` records that the source did not report a value, which is distinct from a reported zero. Totals treat `NULL` as zero, while range filters, percentile thresholds, and the sanity rules skip it, and the derived rates are `NULL` whenever an input is missing or gross load is zero. All foreign keys cascade on delete: removing a facility removes its units and records, and removing a dataset removes its issues and the records it last wrote.

Indexes. Table 3 lists the indexes and the searches they serve. The (`facility_id`, `year`) index was added after profiling (Section 6.4).

Table 3: Secondary and unique indexes.

Index	Serves
<code>facilities(state_code), (name), (county), (nerc_region), (source_category)</code>	Basic-search filters and name sorting
<code>units(facility_id, unit_id) UNIQUE</code>	Unit identity; upsert target
<code>units(facility_id), (primary_fuel), (operating_status)</code>	Joins and unit filters
<code>annual_records(unit_internal_id, year) UNIQUE</code>	Facility-unit-year identity; upsert target; unit history
<code>annual_records(year, co2_mass_tons)</code>	Year filter and CO ₂ ranking
<code>annual_records(facility_id, year)</code>	Per-facility totals, overall or within one year
<code>data_audit_logs(annual_record_id), import_issues(dataset_id)</code>	Flags per record; issues per dataset

Normalisation. The schema is in third normal form apart from two deliberate redundancies. First, `annual_records.facility_id` duplicates information derivable through `units`, so that per-facility aggregates read a single indexed table rather than joining through `units` for each of the 1,619 facilities. Second, the SO₂, NO_x, PM, and mercury controls and the program code are stored on each annual record as reported for that year, as the specification requires, while the unit also keeps the most recent values for display and for records that arrive without them. The per-year values matter: in 16,233 of the 50,947 unit-years, the year’s controls or programs differ from the unit’s current ones.

3.4 Data flow and provenance

Figure 6 shows how data enter and leave the database. The three entry points, the Retrieve page, the Upload page, and the `sync:campd` script, converge on one write path in `src/server/ingest.ts`. Before writing, it loads the stored metrics and per-year attributes of every incoming facility-unit-year and classifies each record as new, changed, or unchanged. It then upserts facilities, units, and annual records, computes the two derived rates, replaces each record’s sanity flags, and records the counts on the dataset row. A record from a file without control columns keeps the controls already stored for that year, or, if the year is new, takes the unit’s current ones. Each import, whether one uploaded file or one retrieved year, runs in a single database transaction, so a failure part-way through leaves nothing behind. Source rows that fail validation or repeat a facility-unit-year are written to `import_issues` with their original values and a reason.

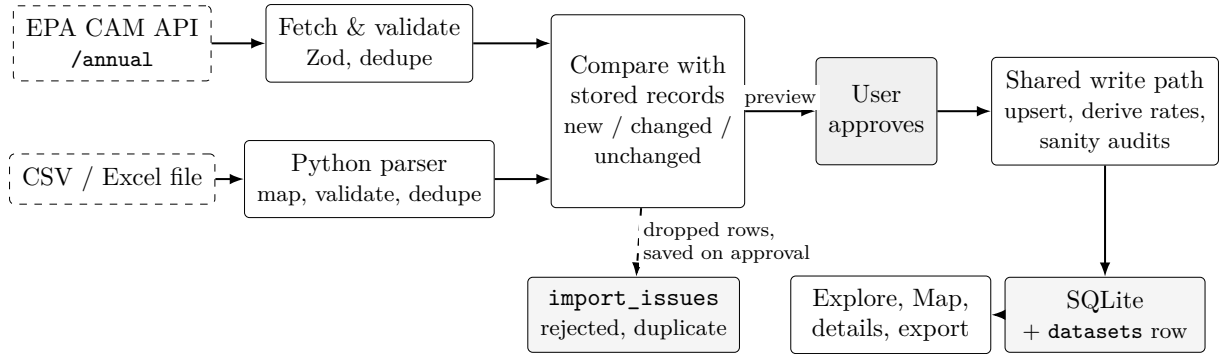


Figure 6: Data flow. Retrieval and upload differ only in how rows are obtained and validated; comparison, approval, writing, and issue logging are shared. Nothing is written before approval, except by the `sync:campd` script, which omits the approval step.

Provenance is recorded at three levels. Each *dataset* stores its source, time, parameters (for API retrievals, the exact query as JSON), file name and archived copy (for uploads), and the counts of rows received, stored, flagged, new, updated, unchanged, and dropped. Each *annual record* references the dataset that last wrote it, which the interface shows as the record’s origin; this is a coarse form of what Buneman et al. term *where-provenance* [3], identifying the source from which each record was copied. Each *issue* preserves a rejected or duplicate source row verbatim. A dataset whose records have all been taken over by a later retrieval is retained and labelled *superseded*, so the retrieval history is never rewritten (Section 7.3).

4 Technical details

4.1 Languages

Three languages are used. **TypeScript** implements the user interface, the server, the query builders, and the command-line scripts; a single typed language across browser and server lets tRPC carry the server’s types into the React components (Section 7). **SQL** is mostly generated by Drizzle, but the window functions, correlated subqueries, and provenance conditions are written as SQL fragments embedded in TypeScript. **Python** reads uploaded files, as the specification requires, using the standard `csv` module and `openpyxl` for Excel workbooks.

4.2 Database system

The database is **SQLite** [6], accessed through the libSQL client (`@libsql/client`). SQLite stores the whole database in one file, `db.sqlite`, which is committed to the repository so that the application runs immediately after installation. It provides every feature the design requires: foreign keys with cascading deletes, unique and composite indexes, upserts (`INSERT ... ON CONFLICT DO UPDATE`), and window functions such as `ROW_NUMBER() OVER (PARTITION BY ...)`, which rank rows within groups in a single pass over sorted input [8] rather than through self-joins or application code. The libSQL client could also connect to a hosted libSQL server by changing `DATABASE_URL`, although the delivered system uses only the local file.

4.3 Frameworks and libraries

Table 4 lists the frameworks and libraries and the role of each.

Table 4: Frameworks and libraries.

Component	Role in <code>epaData</code>
Next.js 16 (App Router)	Application framework: server rendering of the page, route handlers for upload and export, bundling, and the development and production servers.
React 19	User-interface components: pages, tables, the search form, the detail dialog.
tRPC 11	Typed remote procedure calls between browser and server; the 17 <code>facilities.*</code> procedures.
TanStack Query 5	Client-side caching, background refetching, and loading states for tRPC calls.
Drizzle ORM and drizzle-kit	Schema definition in TypeScript, type-checked query building, and generated SQL migrations (seven so far, in <code>drizzle/</code>).
Zod	Validation of every procedure input, of CAMPD API rows, of environment variables, and of the optional language model’s output.
Python 3, <code>csv</code> , <code>openpyxl</code>	Reading, mapping, and validating uploaded CSV and Excel files.
Tailwind CSS 4	Styling with design tokens for light and dark themes.
Leaflet	Two-dimensional map with CARTO/OpenStreetMap tiles.
D3 [2] and TopoJSON	Orthographic globe drawn on a canvas from bundled U.S. and world outlines.
lucide-react, next-themes	Icons; theme switching.
<code>node:test</code> with <code>tsx</code>	Unit-test runner for TypeScript without a separate test framework.
ESLint, Prettier,	Static checks run by <code>npm run validate</code> .
TypeScript compiler	
Tectonic	Builds this report from LaTeX.

4.4 External services

The **EPA CAM API** provides apportioned emissions at annual, monthly, daily, and hourly resolution, together with master-data endpoints listing valid fuel types, unit types, and control technologies. Requests are made only from the server, carry the API key in an `x-api-key` header, and are paged at 500 rows; a cap of 100 pages raises an error rather than silently truncating a result. **OpenRouter**, an optional gateway to hosted language models, serves only as a fallback for description search (Section 5.3); without a key, that feature relies on the rule-based parser alone.

4.5 Scripts and configuration

The npm scripts in `package.json` are the entry points for setup, data maintenance, and quality checks (Table 5).

Table 5: Scripts.

Command	Purpose
<code>npm run dev / build / start</code>	Development server; production build and server.
<code>npm run db:migrate</code>	Applies the SQL migrations in <code>drizzle/</code> to the database named by <code>DATABASE_URL</code> .
<code>npm run db:generate</code>	Generates a migration after a change to <code>src/server/db/schema.ts</code> .
<code>npm run db:seed</code>	Loads facilities and units from CAMPD facility CSV files.
<code>npm run sync:campd</code>	Retrieves annual emissions for one year or a range and stores them (<code>--year</code> , <code>--from</code> , <code>--to</code>).
<code>npm test</code>	Runs the 42 unit tests in <code>src/lib/*.test.ts</code> .
<code>npm run validate</code>	Format check, lint, type check, tests, and production build.
<code>npm run report</code>	Builds this report (<code>report/main.pdf</code>).
<code>scripts/parse_import.py</code> FILE	Prints the validation report of a CSV or Excel file as JSON; called by the upload endpoint.

Configuration is read from environment variables in `.env`, validated at start-up by `src/env.js`, and documented in `.env.example`. Only `DATABASE_URL` is required (`DATABASE_AUTH_TOKEN` applies only to a hosted libSQL server). `CAMPD_API` enables retrieval and the granular views; `OPENROUTER_API_KEY` and `OPENROUTER_MODEL` enable the language-model fallback; and `NEXT_PUBLIC_CARTO_API` improves the map tiles. Variables without the `NEXT_PUBLIC_` prefix are excluded from the browser bundle, which enforces the specification’s rule that the EPA key must not reach users. The README gives the complete installation procedure.

5 Highlighted features

This section presents the six features that most distinguish `epaData`, each with screenshots and an example workflow. All screenshots were taken from the running application on a copy of the committed database.

5.1 Comparing the database with the EPA API before saving

Conventional data loaders write whatever the source returns. `epaData` instead treats retrieval as a merge: it shows what a retrieval *would* change and saves only once the user agrees, answering the question a user has before retrieving, namely whether anything is new.

Example workflow. The committed sample upload (Section 5.2) lowers one stored value, the 2023 CO₂ mass of Trimble County unit 2, from 4,099,943.863 to 4,098,943.863 short tons. After that upload, the user opens Retrieve and selects Kentucky, facility 6071, and 2023:

1. *Already in the database* reports eight stored unit-years for 2023: two last written by the CSV upload and six by an earlier API retrieval.
2. *Preview* fetches the year from the API and compares each row with the stored record (Figure 7): eight rows received, none new, one changed, seven unchanged, none dropped. The Changed tab names the field and both values, “CO₂: 4,098,943.86 → 4,099,943.86”.
3. *Approve & save* stores the year as a new dataset; the history records 0 new, 1 updated, 7 unchanged, and 0 dropped, and the EPA value is restored.

Already in the local database

Stored unit-years for these years and filters, by the dataset that last wrote them. Fuel, unit type, and control match by name, so this approximates what CAMPD will return.

Year %	Stored unit-years %	Last written by %
2023	2	CSV upload - 10/10/2026
2023	6	EPA CAMPD API - 10/10/2026

Preview from the live EPA API

Compared with the stored values for each facility-unit-year. Approving writes new and changed records; unchanged ones are rewritten with the same values, and dropped rows are listed in the invalid-records report.

Received
8
Rows from CAMPD

New
0
Not in the database

Changed
1
Values differ

Unchanged
7
Same as stored

Dropped
0
Invalid or duplicate

New
Changed
Unchanged
Dropped

Year %	Facility %	Unit %	Gross MWh (API) %	CO ₂ t (API) %	Database → API %
2023	Trimble County (#6071)	2	4,614,178	4,099,944	CO ₂ : 4,098,943.86 → 4,099,943.86

Up to 50 rows per category and year are listed; the counts above cover every row.

Nothing is saved until you approve.

Cancel
Approve and save year (1 new or changed)

Past retrievals

New / Updated / Unchanged compare each retrieval with what the database held before it (→ = imported before this was tracked). Superseded datasets had all their records taken over by a later retrieval; they are kept as history.

Year %	Source %	Parameters %	Imported %	Received %	Stored %	New %	Updated %	Unchanged %	Dropped %	Status %
2026	EPA CAMPD API	All units	10/10/2026, 9:59:21 AM	3,959	3,959	0	3,959	0	0	Active
2025	EPA CAMPD API	All units	10/10/2026, 9:59:19 AM	3,989	3,989	0	3,989	0	0	Active
2024	EPA CAMPD API	All units	10/10/2026, 9:59:16 AM	4,024	4,024	0	4,024	0	0	Active
2023	EPA CAMPD API	All units	10/10/2026, 9:59:14 AM	4,090	4,090	0	4,090	0	0	Active
2022	EPA CAMPD API	All units	10/10/2026, 9:59:12 AM	4,131	4,131	0	4,131	0	0	Active
2021	EPA CAMPD API	All units	10/10/2026, 9:59:10 AM	4,178	4,178	0	4,178	0	0	Active
2020	EPA CAMPD API	All units	10/10/2026, 9:59:08 AM	4,254	4,254	0	4,254	0	0	Active

Figure 7: Retrieval preview for Trimble County, 2023: the database panel (top), the API comparison with one changed record and its old and new value, and the approval button.

Implementation. Preview and save share one fetch function, `fetchCampdAnnual`, which pages through the API, validates every row with Zod, removes rows that repeat a facility-unit-year (including across pages), and writes nothing. The comparison keys each record by its natural key (facility ID, unit ID, year) and compares the seven metrics and five per-year attributes (Listing 1). Values that differ only by floating-point noise count as equal, so re-reading an unchanged CAMPD figure never registers as a change, while a missing value differs from a reported zero. Rows that fail validation or repeat a key are *dropped* but not lost: saving writes them to `import_issues` with the Zod message as the reason, so the invalid-record report covers API datasets as well as uploads. The save step re-fetches the year rather than trusting the preview, fetches it completely before the first write, and then writes it in one transaction, so neither a network nor a database failure leaves a partial dataset; a failed attempt is kept in the retrieval history with its error.

```

1 /** Natural key of a unit-year record, shared by uploads, CAMPD rows, and stored records. */
2 export const recordKey = (facilityId: number, unitId: string, year: number | string) =>
3   `${facilityId}:${unitId}:${year}`;
4
5 /** Equal within floating-point noise (relative 1e-9); null (not reported) equals only null.
6   */
7 const sameMetric = (a: number | null, b: number | null) =>
8   a === null || b === null
9     ? a === b
10     : Math.abs(a - b) <= 1e-9 * Math.max(1, Math.abs(a), Math.abs(b));
11
12 /** New when nothing is stored; otherwise every differing metric, and every per-year
13   attribute the
14   * incoming record carries (uploads without control columns don't compare them). */
15 export function diffRecord(incoming: ComparableRecord, stored?: ComparableRecord) {
16   if (!stored) return { status: "new", changes: [] };
17   const changes = [

```

```

16 ...TOTAL_KEYS.filter((k) => !sameMetric(incoming[k], stored[k])),
17 ...RECORD_ATTRIBUTE_KEYS.filter(
18   (k) => incoming[k] !== undefined && (incoming[k] ?? null) !== (stored[k] ?? null),
19 ),
20 ].map((field) => ({ field, database: stored[field] ?? null, incoming: incoming[field] ??
   null }));
21 return { status: changes.length ? "changed" : "unchanged", changes };
22 }

```

Listing 1: Record comparison (src/lib/record-diff.ts), shared by retrievals and uploads.

5.2 Upload validation and the data-quality report

The upload path implements all eleven steps of the specification. Its distinguishing property is that no row is lost: every row is stored, stored and flagged, or listed in the data-quality report with a reason.

Validation rules. scripts/parse_import.py normalises each header by lower-casing it and removing punctuation, so “Facility ID”, facility_id, and facilityId all match the same alias list. It then maps the file to one of two schemas: annual emissions (facility ID, unit ID, year, and metrics) or facilities and units (no year). Each cell is converted by a column-specific rule (Listing 2): identifiers and years must be whole numbers within bounds, coordinates must lie within $\pm 90/\pm 180$ degrees, metrics must be non-negative, and states must be valid U.S. state or territory codes. A row with any failing cell is rejected with every reason listed; a row whose facility-unit-year already appeared earlier in the file is skipped as a duplicate with a reference to the first occurrence.

```

1 def parse_value(col, raw):
2     """Convert a non-empty cell to its database value, or raise ValueError with the reason.
   """
3     if col == "stateCode":
4         if raw.upper() not in US_STATES:
5             raise ValueError(f"Unrecognized US state or territory '{raw}'")
6         return raw.upper()
7     if col not in BOUNDS:
8         return raw
9     lo, hi, whole = BOUNDS[col]          # e.g. year: (1980, 2100, True)
10    try:
11        n = float(raw.replace(",", ""))
12    except ValueError:
13        n = math.nan
14    if not math.isfinite(n):
15        raise ValueError(f"Expected a number, got '{raw}'")
16    if whole and not n.is_integer():
17        raise ValueError(f"Expected a whole number, got '{raw}'")
18    if n < lo or (hi is not None and n > hi):
19        raise ValueError(f"Out of range ({lo} to {hi})" if hi is not None else f"Must be at
   least {lo}")
20    return int(n) if whole else n

```

Listing 2: Cell validation in scripts/parse_import.py (abridged).

Example workflow. The repository includes a test file, samples/annual-emissions-sample.csv, with an identical Excel copy. It contains 25 rows of real CAMPD data for Kentucky, mostly from 2014, a year the committed database does not cover; five rows were deliberately altered, and one unaltered row trips a sanity rule. Uploading it produces the report

in Figure 8: 21 valid rows (19 new, one changed, one unchanged); three rejected rows, with a missing unit ID (row 12), the state code “KZ” (row 18), and a negative CO₂ mass (row 19); one duplicate, row 25 repeating row 4; and one sanity flag on row 16, Robert Reid unit RT in 2014, which reported 38,801.9 MMBtu of heat input but left the CO₂ mass blank in unmodified EPA data. The blank is stored as NULL and raises the CO₂_NOT_REPORTED warning (Section 5.5). Repeats within the file and records already in the database appear on separate tabs, so each tab’s count matches its summary tile. After approval, the Unit-years tab filtered to origin “File upload” lists exactly the 21 stored unit-years, and the invalid-record download contains the three rejected rows, the duplicate, and the flagged record, each with its original columns.

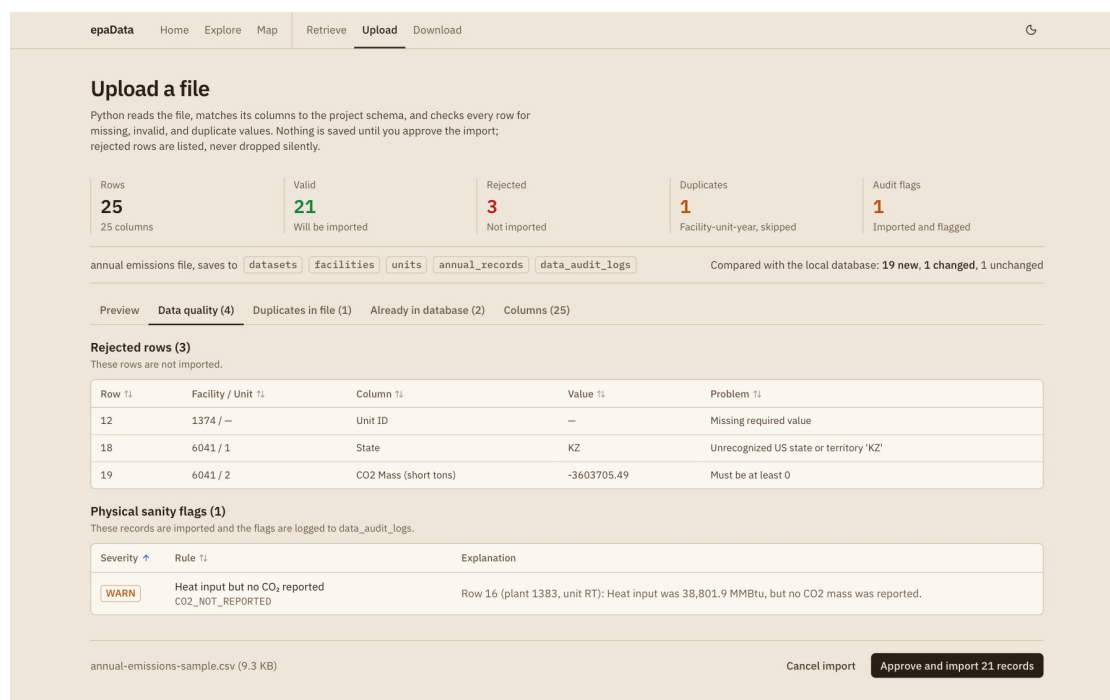


Figure 8: Upload report for the sample file, Data Quality tab: counts, the database comparison, the three rejected rows with their reasons, and the physical-sanity flag.

5.3 Description-based search

The rubric asks for a search that understands requests such as “Find coal units in Kentucky with high CO₂ emissions.” A large language model could translate such a sentence, but it would make the core search depend on a network service and an API key, and its output may vary between runs. epaData therefore uses a deterministic, rule-based parser (`src/lib/describe-search.ts`) and consults a language model only for words the parser could not use, and only if a key is configured.

Algorithm. The parser performs entity extraction rather than template matching, so word order does not matter. It operates in two passes (Listing 3).

```

vocabulary <- filter values from the database (states, fuels, unit types, controls,
          counties, statuses) + synonyms ("scrubber" -> FGD, "gas" -> Natural Gas,
          state names -> codes) + metric, comparator, ranking, and view words

pass 1: tokens -> items
  for each position, take the LONGEST phrase (up to 6 words) found in the vocabulary
    ("west virginia" beats "virginia"; "natural gas" beats "gas")
  two-letter state codes count only in capitals ("IN", not "in")

```

```

counties need the word "county"; names follow "owned by" / "named"
numbers: 500,000 | 500k | 1.2M | 5e5; four-digit years
unknown filter value within edit distance 1 (2 for long words) -> fuzzy match
"not X" -> reported as not understood, never inverted
anything else that is not a stopword -> not understood

pass 2: bind items
filters -> filter values; "units" / "plants" -> which table to show
comparator + number -> nearest metric word, looking forward then back
    ("over 500k tons of CO2", "CO2 over 500k", "under 50 tons SO2")
year-like numbers -> year, or year range ("since 2020", "2015-2020")
"top / highest / most [N]" -> sort descending, keep N (default 10, or 1 if singular)
"lowest / least / cleanest [N]" -> sort ascending; "in each state" -> rank per state
"high / low <metric>" without a number -> qualitative term, resolved on the server

```

Listing 3: Outline of the description parser.

Typo tolerance uses the optimal-string-alignment variant of edit distance, which counts insertions, deletions, substitutions, and adjacent transpositions, the four error types Damerau identified as the bulk of spelling mistakes [5]. It applies only to filter values of five or more letters, so short words are never “corrected” into something else.

Resolving “high” and “low”. A qualitative term has no fixed numeric meaning: 500,000 tons of CO₂ is high for a gas turbine and low for a large coal unit. The server therefore resolves “high m ” and “low m ” to the 75th and 25th percentiles of metric m over the unit-years matching all *other* filters. With the n matching values in ascending order, $x_{(0)} \leq \dots \leq x_{(n-1)}$, the thresholds are the order statistics

$$t_{\text{high}} = x_{(\lfloor 0.75(n-1) \rfloor)}, \quad t_{\text{low}} = x_{(\lfloor 0.25(n-1) \rfloor)}, \quad (1)$$

each obtained with a single `ORDER BY  $m$  LIMIT 1 OFFSET  $k$` query. This is the simplest of the sample-quantile definitions compared by Hyndman and Fan [7]: where their common default interpolates between neighbouring order statistics, epaData takes the lower one, so the threshold is computed from a value present in the data rather than an interpolated one. Unit-years that do not report the metric are excluded, as they have no value to rank. The threshold is then rounded to three significant figures for readability, becomes an ordinary minimum or maximum filter, and sorts the table by that metric. For the rubric sentence, the 385 of 416 Kentucky coal unit-years that report CO₂ give $t_{\text{high}} = 2,720,000$ tons, which selects 97 unit-years (Figure 9); the interface states the rule beside the result (“high = top 25% of unit-years matching the other filters”).

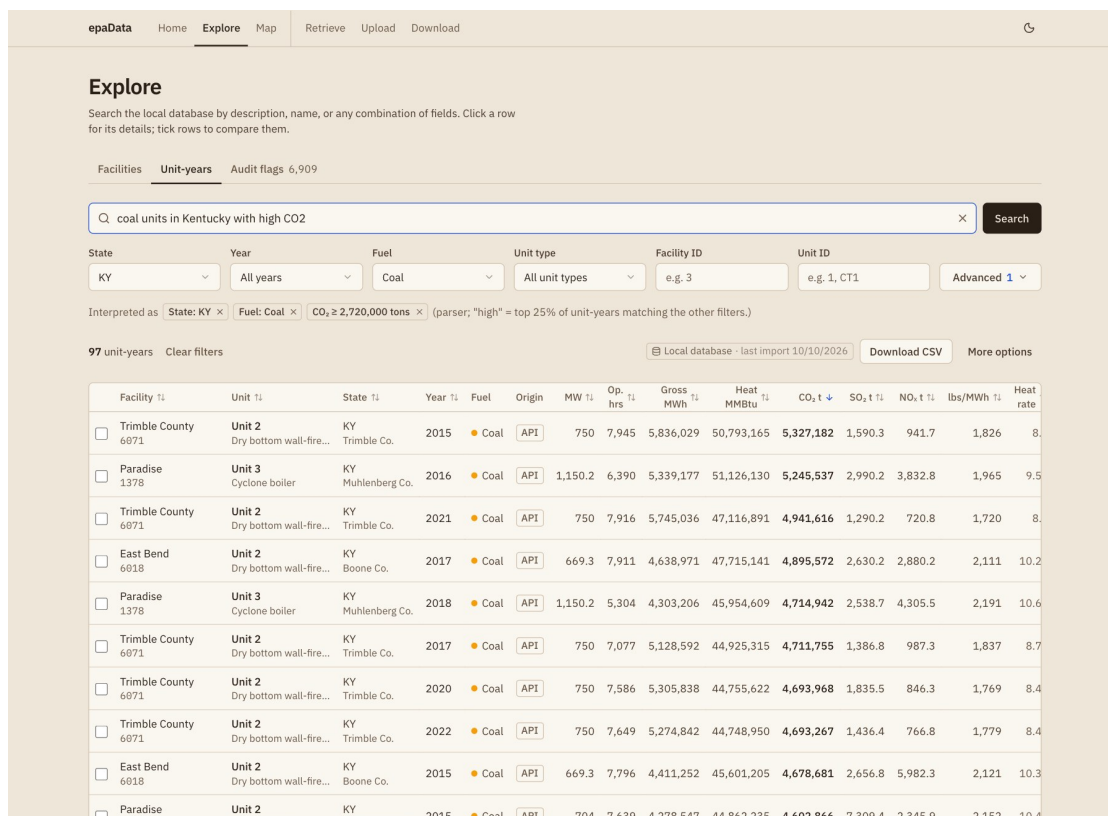


Figure 9: Description search on the committed database. The sentence becomes three removable filter chips; the note explains the threshold; 97 unit-years match, sorted by CO₂.

Language-model fallback. When words remain unrecognised and `OPENROUTER_API_KEY` is set, `src/server/llm-search.ts` sends the sentence, the parser’s interpretation, and the leftover words to an OpenRouter model, requesting structured JSON whose schema enumerates the database’s own filter values. The model may only *add* filters the parser did not set; every returned value is re-validated with Zod against the vocabulary; the call times out after 8 seconds; and any failure falls back to the parser’s result with a note. The model therefore cannot introduce a value absent from the database, and the search remains fully functional without it.

Worked examples. Table 6 lists ten descriptions, the filters the parser produced, and the number of rows each returns on the committed database. None required the language model.

Table 6: Description search on the committed database (parser only).

Description	Interpreted as	Rows
Find coal units in Kentucky with high CO2 emissions.	Units; KY; Coal; CO ₂ ≥ 2,720,000; sort CO ₂ ↓	97
coal units in Kentucky in 2025 with CO2 over 500,000 tons	Units; KY; Coal; 2025; CO ₂ ≥ 500,000	25
top CO2-emitting facility in each state in 2024	Facilities; 2024; first 1 per state by CO ₂ ↓	51
top 20 units by gross generation in 2025	Units; 2025; first 20 by gross load↓	20
units with the lowest NOx emissions in 2025	Units; 2025; first 10 by NO _x ↑	10
gas units in TX under 50 tons SO2	Units; TX; Natural Gas; SO ₂ ≤ 50	3,857

Description	Interpreted as	Rows
operating gas units in Texas with low NO _x since 2020	Units; TX; Natural Gas; Operating; year ≥ 2020 ; NO _x ≤ 9.37 ; sort NO _x ↑	660
plants owned by Duke Energy with scrubbers	Facilities; text “Duke Energy”; SO ₂ control FGD	11
units in IN with heat input above 5M MMBtu in 2025	Units; IN; 2025; heat input $\geq 5,000,000$; sort heat input↓	42
Kentucky coal units that are not retired	Units; KY (typo corrected); Coal; <i>not understood: “not retired”</i>	416

5.4 Multi-constraint search and ranking in SQL

The specification’s example, “coal-fired units in Kentucky in 2025 with annual CO₂ emissions greater than 500,000 short tons”, is a conjunction of conditions on the facility (state), the unit (fuel), and the unit-year (year and CO₂). The query builder has one function per level and combines their output with AND (Figure 10). Fuel, unit-type, and control filters match by substring, because CAMPD stores combined values such as “Coal, Pipeline Natural Gas” or “Wet Lime FGD|Wet Limestone”; operating status matches by prefix, so that “Retired” does not also match “Operating (Retired 04/15/2016)”.

Q Describe what you want, or type a name: "coal units in Kentucky with high CO2" Search

State: KY Year: 2025 Fuel: Coal Unit type: All unit types Facility ID: e.g. 3 Unit ID: e.g. 1, CT1 Advanced 1

County: All counties Grid region: All grids Secondary fuel: Any SO₂ control: Any NO_x control: Any

PM control: Any Operating status: Any Audit flag: Any (or none) Flag severity: Any Record origin: Any

Ranges per unit-year (minimum and maximum, inclusive)

Reporting year (yr): min max Operating time (hr): min max Gross load (MWh): min max

Heat input (MMBtu): min max CO₂ (tons): 500000 max SO₂ (tons): min max

NO_x (tons): min max

Ranking: All rows Group: Overall Ranks follow the table sort: sort a column descending for Top-N, ascending for Bottom-N. Each row is one unit in one reporting year.

Filters: State: KY Fuel: Coal Year: 2025 CO₂ $\geq 500,000$ tons

25 unit-years Clear filters Local database - last import 10/10/2026 Download CSV More options

Facility ID	Unit ID	State	Year	Fuel	Origin	MW	Op. hrs	Gross MWh	Heat MMBtu	CO ₂ t	SO ₂ t	NO _x t	lbs/MWh	Heat rate
Trimble County 6071	Unit 2 Dry bottom wall-fire...	KY Trimble Co.	2025	Coal	API	750	7,053	4,808,516	40,531,843	4,250,980	1,494	777.4	1,768	8.43
H L Spurlock 6041	Unit 2 Tangentially-fired	KY Mason Co.	2025	Coal	API	610	7,374	3,465,337	35,746,445	3,667,586	1,329.3	1,534.3	2,117	10.32
Trimble County 6071	Unit 1 Tangentially-fired	KY Trimble Co.	2025	Coal	API	566.1	7,711	3,758,564	33,145,031	3,400,682	2,619.7	1,187.6	1,810	8.82
Mill Creek 1364	Unit 4 Dry bottom wall-fire...	KY Jefferson Co.	2025	Coal	API	543.6	7,257	3,135,113	30,086,675	3,083,035	792.3	1,356.5	1,967	9.6
Ghent 1356	Unit 2 Tangentially-fired	KY Carroll Co.	2025	Coal	API	556.4	8,295	3,216,587	29,945,706	3,072,430	5,660.5	3,667.8	1,910	9.31

Figure 10: Multi-constraint search: KY, Coal, 2025, CO₂ $\geq 500,000$ t. The chips, the open *Advanced* panel with its range inputs, and the first rows of the 25 results.

Ranking is implemented with a window function. Listing 4 shows, simplified, the SQL that `rankedUnitYears` generates for “the highest-CO₂ coal unit-year in each state in 2025”. The inner query numbers the matching rows in sort order, restarting for each state when ranking per state, and the outer query keeps the first *N* ranks. The page of rows, the total count, and the CSV export all derive from this one subquery and therefore always agree. `IS NULL` in the ordering places missing values last in either direction, and the record ID breaks ties so that

paging is stable.

```

1 SELECT * FROM (
2   SELECT ar.id, f.name, f.state_code, u.unit_id, ar.year, ar.co2_mass_tons, d.source AS
      origin,
3     ROW_NUMBER() OVER (
4       PARTITION BY f.state_code                                -- only when ranking per state
5       ORDER BY ar.co2_mass_tons IS NULL, ar.co2_mass_tons DESC, ar.id
6     ) AS rank
7 FROM annual_records ar
8 JOIN units u          ON ar.unit_internal_id = u.id
9 JOIN facilities f      ON ar.facility_id = f.id
10 LEFT JOIN datasets d ON ar.dataset_id = d.id
11 WHERE lower(u.primary_fuel) LIKE '%coal%'                    -- unit condition (contains)
12    AND ar.year = 2025                                         -- record conditions
13 ) AS ranked
14 WHERE rank <= 1                                              -- keep the first N
15 ORDER BY state_code, rank
16 LIMIT 25 OFFSET 0;                                         -- one page

```

Listing 4: Ranking query generated for a per-state top-1 search (simplified).

The Facilities view ranks plants by totals computed with correlated subqueries, for example the sum of CO₂ over a facility’s records in the selected year. Unit and unit-year filters retain a facility if at least one of its unit-years matches, expressed as an IN subquery so that the facility row is not duplicated. Figure 11 shows the per-state ranking for 2024.

The screenshot shows the EPA Facilities view interface. At the top, there is a search bar with the text "Describe what you want, or type a name: 'coal units in Kentucky with high CO2'". Below the search bar, there are filters for State (All states), Year (2024), Fuel (All fuels), Unit type (All unit types), Facility ID (e.g. 3), and Unit ID (e.g. 1, CT1). There is also an "Advanced 2" button. Below the filters, there is a "Filters" section with "Year 2024" and "First 1 per state" selected. The main table displays 51 facilities, sorted by CO₂ emissions in 2024. The table has columns for Facility ID, ID, State, Primary fuel, Role, grid, Capacity, and CO₂, 2024. The facilities listed include Healy Power Plant, James H Miller Jr, Independence, Springerville Generating Station, High Desert Power Project, Comanche (470), Lake Road Generating Company, GSA Central Heating, Hay Road, Crystal River, Bowen, Walter Scott Jr. Energy Center, and Langley Gulch Power Plant.

Facility ID	ID	State	Primary fuel	Role, grid	Capacity	CO ₂ , 2024
1. Healy Power Plant	6288	AK	Coal	Baseload Plant	89 MW	—
Golden Valley Electric Association, Inc.		Denali Borough...		—	2 units	—
1. James H Miller Jr	6802	AL	Coal	Baseload Plant	2,822 MW	20,798,074 t
Alabama Electric Cooperative, Inc., Alabama Power ...		Jefferson Co.		SERC	4 units	High Carbon Coal
1. Independence	6641	AR	Coal	Baseload Plant	1,700 MW	6,736,359 t
Arkansas Electric Cooperative Corporation, City of Jo...		Independence ...		SPP	2 units	High Carbon Coal
1. Springerville Generating Station	8223	AZ	Coal	Baseload Plant	1,765.8 MW	6,552,258 t
Tucson Electric Power Company		Apache Co.		WECC	4 units	High Carbon Coal
1. High Desert Power Project	55518	CA	Pipeline Natural Gas	Baseload Plant	1,521 MW	1,987,693 t
High Desert Power Project, LLC		San Bernardino...		—	3 units	Low Carbon CCGT
1. Comanche (470)	470	CO	Coal	Baseload Plant	1,486 MW	6,520,388 t
Public Service Company of Colorado		Pueblo Co.		WECC	3 units	High Carbon Coal
1. Lake Road Generating Company	55149	CT	Pipeline Natural Gas	Baseload Plant	840 MW	2,428,509 t
Lake Road Generating Company, LP		Windham Co.		—	3 units	Intermediate Peaker
1. GSA Central Heating	888804	DC	Pipeline Natural Gas	Industrial Cogen	10.8 MW	—
GSA		District of Colu...		—	3 units	—
1. Hay Road	7153	DE	Pipeline Natural Gas	Baseload Plant	2,053.5 MW	924,458 t
Calpine Mid-Atlantic Generation, LLC, Calpine Operat...		New Castle Co.		—	6 units	Intermediate Peaker
1. Crystal River	628	FL	Coal	Baseload Plant	5,143 MW	8,474,913 t
Duke Energy Florida, LLC		Citrus Co.		FRCC	8 units	Intermediate Peaker
1. Bowen	703	GA	Coal	Baseload Plant	3,498.6 MW	11,428,296 t
Georgia Power Company		Bartow Co.		SERC	4 units	High Carbon Coal
1. Walter Scott Jr. Energy Center	1082	IA	Coal	Baseload Plant	1,648.9 MW	6,991,122 t
Atlantic Municipal Utilities, Central Iowa Power Coop...		Pottawattamie ...		—	4 units	High Carbon Coal
1. Langley Gulch Power Plant	57028	ID	Pipeline Natural Gas	Baseload Plant	318 MW	754,965 t
Idaho Power Company		Payette Co.		—	1 unit	Low Carbon CCGT

Figure 11: Group-and-rank: the top CO₂-emitting facility in each state in 2024 (51 rows).

5.5 Physical-sanity audits

Validation detects malformed values but not values that are well-formed yet physically implausible. epaData therefore applies four rules to every annual record it writes, whatever its source.

Let H be heat input (MMBtu), G gross load (MWh), T operating time (hours), and C the CO₂ mass (short tons). The derived rates stored with each record are

$$\text{heat rate} = \frac{H}{G} \text{ MMBtu/MWh}, \quad \text{CO}_2 \text{ intensity} = \frac{2000 C}{G} \text{ lb/MWh}, \quad (2)$$

both undefined (stored as NULL) when $G = 0$ or an input is not reported. Writing $\emptyset$ for a value that was not reported, the rules, with thresholds defined once in `AUDIT_THRESHOLDS`, are:

$$\text{ZERO_EMISSIONS_HIGH_HEAT (ERROR)} : H > 1000 \wedge C = 0, \quad (3)$$

$$\text{CO2_NOT_REPORTED (WARN)} : H > 1000 \wedge C = \emptyset, \quad (4)$$

$$\text{PHANTOM_GENERATION (ERROR)} : G > 0 \wedge T = 0, \quad (5)$$

$$\text{EXTREME_HEAT_RATE (WARN)} : H/G < 5 \vee H/G > 25. \quad (6)$$

A comparison involving $\emptyset$ is false, so a rule never fires on a missing input. The first rule reflects that burning fossil fuel always produces CO₂, so a reported zero is implausible, whereas the second records a reporting gap; the third reflects that a unit cannot generate without operating; the fourth bounds the heat rate by the range of real fossil plants, from about 6.5 MMBtu/MWh for modern combined-cycle units to 15–18 MMBtu/MWh for old peaking units. Flagged records are retained and appear in the Audit flags tab, the unit and facility details, and the invalid-record export. In the committed database the rules raise 6,909 flags on 6,432 records: 6,105 records with heat input but no reported CO₂ and 804 extreme heat rates, all warnings. Most of the former are gas- and oil-fired units, and 2,637 of them also report no gross load. No record reports a CO₂ mass of exactly zero alongside significant heat input, so no ERROR is raised (Section 7.3).

5.6 Provenance and export

Every stored figure can be traced to its source. The unit details (Figure 12) list each reporting year with the dataset that wrote it; the Unit-years table shows the same information as an *Origin* badge; and the *Origin* filter restricts any search to API or uploaded records. The Download page (Figure 13) exports the provenance table itself, one row per dataset with its source, date, query parameters as JSON, file name, and the received, stored, flagged, new, updated, unchanged, and dropped counts. Unit-year exports include `dataset_id`, `origin`, and `dataset_imported_at` for every row. All exports are written by one CSV function that quotes fields according to RFC 4180.

Year	Op. Hours	Gross MWh	Heat Input	Steam kWh	CO ₂ t	SO ₂ t	NO _x t	HHV MWh	HHV/MMBtu	Controls, programs	Source
2026	4,075	1,467,467	14,302,635	—	1,467,448	486.8	423.6	2,000	9.75	SO ₂ , NO _x , PM, Hg, APT...	API, 10/10/2026
2025	7,264	2,797,461	26,862,483	—	2,798,366	954.3	901.7	2,000	9.75	SO ₂ , NO _x , PM, Hg, APT...	API, 10/10/2026
2024	7,846	3,133,799	30,246,939	—	3,103,540	895.6	1,009.9	1,981	9.65	SO ₂ , NO _x , PM, Hg, APT...	API, 10/10/2026
2023	7,280	2,899,444	27,953,962	—	2,866,075	770.8	716.8	1,978	9.64	SO ₂ , NO _x , PM, Hg, APT...	API, 10/10/2026
2022	7,364	2,937,709	28,253,249	—	2,896,784	1,322.5	750.7	1,974	9.62	SO ₂ , NO _x , PM, Hg, APT...	API, 10/10/2026
2021	6,696	2,929,928	27,871,685	—	2,893,635	1,399.7	978.4	1,952	9.51	SO ₂ , NO _x , PM, Hg, APT...	API, 10/10/2026
2020	6,705	2,886,021	27,706,603	—	2,863,217	1,313.7	856.5	1,984	9.67	SO ₂ , NO _x , PM, Hg, APT...	API, 10/10/2026
2019	7,477	2,930,256	29,113,299	—	2,987,025	821.4	881.3	2,039	9.94	SO ₂ , NO _x , PM, Hg, APT...	API, 10/10/2026
2018	7,818	3,223,392	32,046,646	—	3,287,986	864.7	822.4	2,033	9.91	SO ₂ , NO _x , PM, Hg, APT...	API, 10/10/2026
2017	7,731	3,348,449	32,674,939	—	3,352,449	969.4	1,066.1	2,002	9.76	SO ₂ , NO _x , PM, Hg, APT...	API, 10/10/2026
2016	7,767	3,307,406	31,973,526	—	3,285,484	1,091.3	1,123.3	1,984	9.67	SO ₂ , NO _x , PM, Hg, APT...	API, 10/10/2026
2015	6,640	2,756,519	27,327,354	—	2,803,785	1,360.1	1,087.4	2,034	9.91	SO ₂ , NO _x , PM, Hg, APT...	API, 10/10/2026
2014	6,330	3,512,997	34,265,461	—	3,515,636	1,734.7	1,263.8	2,002	9.76	SO ₂ , NO _x , PM, Hg, APT...	Updated, 10/10/2026

Figure 12: Unit details: Ghent unit 1 after the sample upload, with 13 reporting years (2014 from the upload, 2015–2026 from the API).

Figure 13: Download page: data type, dataset or filters, and format.

6 Testing and results

6.1 Approach

The system was tested at four levels:

1. **Static checks.** `npm run validate` runs Prettier, ESLint, the TypeScript compiler in strict mode, the unit tests, and a production build, and was kept passing after every development stage. Because database rows, procedure inputs, and component props share types, type checking catches a large class of errors early.
2. **Unit tests.** 42 tests using Node’s built-in runner cover the logic in `src/lib` and the Python upload parser, which the tests run as a subprocess on generated and committed files.
3. **Acceptance tests.** For each search in the specification, the application’s result, obtained through its HTTP API, was compared with an independent SQL query written directly against the database.
4. **End-to-end tests.** The upload, retrieval, and export flows were exercised over HTTP, and the interface was driven by a scripted headless browser, which also produced the screenshots in this report. These runs used copies of `db.sqlite`, leaving the committed data unchanged.

6.2 Unit tests

Table 7 summarises the unit tests by file. Each test contains several assertions; the description-search tests, for example, check about sixty phrasings, including three word orders of one request that must yield identical filters.

Table 7: Unit tests (`npm test`: 42 tests, all passing).

File	Tests	What is checked
<code>describe-search.test.ts</code>	12	Rubric and specification sentences; word-order permutations; ranges, units of measure, and <i>between</i> ; top- <i>N</i> , bottom- <i>N</i> , and per-state ranking; years and year ranges; synonyms, controls, and statuses; state names, codes, and ambiguous lower-case codes; owner names; typos; negation and unknown words; number formats.
<code>domain-utils.test.ts</code>	14	Emissions arithmetic with missing generation; not-reported values in rates, totals, and the four sanity rules; fuel metadata; control rules; reporting-period dates and EPA’s published-through quarter; yearly roll-ups; retrieval filters; explorer state round-trip through the URL and rejection of invalid parameters; option splitting; sorting.
<code>data-import.test.ts</code>	8	Extension and size checks; rejected rows, duplicates, and sanity flags on a generated file; daily files, whose repeated unit rows are not duplicates; column mapping; refusal of files without IDs; Excel reading; a real CAMPD facility file (skipped when absent); the committed sample files (Section 5.2).
<code>record-diff.test.ts</code>	5	New, unchanged (including floating-point noise), and changed records with both values; not reported versus zero; per-year controls; counting and natural keys.
<code>csv.test.ts</code>	3	RFC 4180 quoting of commas, quotes, and line breaks; CRLF line endings; export URLs.

6.3 Representative test cases

Table 8 lists representative acceptance and end-to-end cases. Each expected value comes from an independent source, usually a hand-written SQL query or the documented contents of the sample file; all counts refer to the committed database unless stated otherwise.

Table 8: Representative test cases.

Input	Expected (source)	Actual	Pass
Units: KY, Coal, 2025, CO ₂ ≥ 500,000	25 unit-years (SQL)	25	✓
Facilities: same filters	8 facilities (SQL)	8	✓
Facilities: 2024, top 1 per state by CO ₂	51, one per state with 2024 data (SQL)	51	✓
Units: facility 3, unit 1, 2015–2025	11 years (SQL)	11	✓
Units: operating status “Retired”	31 unit-years, prefix match (SQL)	31	✓
Describe: “Find coal units in Kentucky with high CO2 emissions.”	P75 of the 385 reported values = 2,721,500 → 2,720,000; 97 rows (SQL)	2,720,000; 97 rows	✓
Describe: same request in three word orders	Identical filters	Identical	✓
Describe: “Kentucky coal units that are not retired”	KY, Coal; “not retired” reported	As expected	✓
CSV export of the description search	97 data rows	97	✓
2025 dataset (CAMPD API)	3,989 records, 415 flagged (SQL)	3,989; 415	✓
Re-sync 2015–2026 after migration 0006	50,947 records; blanks stored as NULL; 12 datasets	50,947; 12	✓
Import that fails after its first writes	Nothing stored (rollback)	Counts unchanged	✓
Upload without control columns over a stored year (facility 136, unit 1, 2022) and a new year (2013)	2022 keeps its own controls; 2013 takes the unit’s	As expected	✓
Upload the sample file (samples/, CSV)	25 rows: 21 valid (19 new, 1 changed, 1 unchanged), 3 rejected, 1 duplicate, 1 flag (sample README)	As expected	✓
Upload the Excel copy	Same report as the CSV	Same	✓
Approve the sample upload	21 records stored; 4 issues; Origin = Upload lists 21	21; 4; 21	✓
Upload the sample again	21 unchanged, 0 new	21; 0	✓
Retrieve KY, facility 6071, 2023 after the upload	8 received, 1 changed (CO ₂ restored), 7 unchanged	As expected	✓
Retrieve preview, then cancel	No dataset written	Dataset count unchanged	✓
Upload a CAMPD daily file (359,723 rows)	Attributes of 3,953 units; no annual records; no duplicates	As expected	✓
Upload a .pdf file / an empty file	Rejected before parsing	Rejected with reason	✓
Search the client and server bundles for the EPA key	0 occurrences	0	✓

6.4 Performance

Response times were measured on an Apple-silicon laptop against a production build, as the median of three requests after one warm-up (Table 9).

Table 9: Measured response times on the full database (50,947 unit-years).

Operation	Time
Multi-constraint unit search (KY, Coal, 2025, CO ₂ ≥ 500,000), one page	6 ms
Description search (parse and percentile threshold)	29 ms
Facilities, default view, one page	63 ms
Units, no filters (count over all 50,947 rows), one page	237 ms
Units filtered by a per-year control (SO ₂ control contains “FGD”), one page	24 ms
Top facility per state in 2024, <i>before</i> the (facility_id, year) index	9,500 ms
Top facility per state in 2024, <i>after</i> the index	22 ms
CSV export of all 50,947 unit-years	1.1 s
Preview of a full year from the EPA API (3,989 rows, 8 pages)	2.3 s

The two ranking rows record a defect found during testing. Every facility-level query with a year filter took 4–10 seconds, whereas the same query without a year took 90 ms. The query plan showed the cause: the per-facility totals are correlated subqueries of the form `WHERE facility_id = ? AND year = ?`, and with only single-column statistics the planner chose the (year, co2_mass_tons) index, scanning all four thousand records of the year once for each of the 1,619 facilities. A composite index on (facility_id, year), added as migration 0005, allows the planner to match both conditions; the query became over four hundred times faster with identical results.

6.5 Known issues

A final audit of this report against the code found several defects, all of which were corrected. Uploading a CAMPD daily file, which has no year column, keyed each row by facility and unit alone, so every day after the first was reported as a duplicate: a 359,723-row file produced 355,770 spurious issue rows. Sub-annual rows are now identified by their date and hour, and a regression test covers the case. Imports were not transactional, so a database error part-way through could leave a partial dataset; each import now commits or rolls back as a whole, which a deliberately failing import confirmed. Missing metrics were stored as zero, which turned 6,105 unreported CO₂ values into zero-emission errors; they are now stored as NULL and flagged as warnings (Section 7.3). In the interface, the comparison dialog labelled gross load as “Net Generation”; the upload report counted rows already in the database as duplicates; the map omitted facilities without coordinates without saying so; and the shared dialog component lacked dialog semantics. It now declares `role="dialog"`, moves focus into the dialog, keeps Tab within it, closes on Escape, and restores focus on close. These behaviours were verified in a headless browser. A final review found three further defects. The facility table derived each plant’s operating role from operating hours summed over every unit and every year, so a peaking station with a dozen years of records was labelled a baseload plant; the role now uses the busiest unit’s hours in the latest year in scope, as the facility details do. The map’s fuel legend counted plants by fuel class but filtered by a single fuel name, so the oil chip advertised 108 plants and showed 22; the legend now filters by the same class it counts. Finally, a one-pixel overflow gave every tab bar a spurious scrollbar.

One issue remains: the language-model fallback was tested only on its failure paths (no key, invalid key, timeout), since no OpenRouter key was available during testing.

6.6 Assessment against the objectives

Table 10 assesses each Phase 1 objective from Section 1.

Table 10: Assessment of the Phase 1 objectives.

	Objective	Verdict	Evidence
O1	Web system with front end and database	Met	Sections 2 and 3.
O2	Routes, templates, forms, interactive components	Met	With Next.js and React in place of Flask and Jinja (Section 7).
O3	Relational database with SQLite and an ORM	Met	Six tables, keys, cascades, 14 indexes; Drizzle in place of SQLAlchemy.
O4	Retrieve public data or import files	Met	CAM API retrieval with preview; CSV and Excel upload.
O5	Validate, clean, transform, store	Met	Upload and retrieval validation; issues kept; sanity audits.
O6	Search, filter, sort, paginate	Met	Basic, multi-constraint, ranking, and description search; all results sortable and paginated; Table 8.
O7	Export data and results	Met	Six CSV export types.
O11	Test and deploy	Partly met	Tested as described above. Runs locally from the README; not deployed to a public host.
O12	Document sources, assumptions, equations, units, limitations	Met	README, DATABASE_BREAKDOWN.md, this report.

Every Phase 1 data-management requirement is implemented and tested, and every specification example returns the same result as an independent SQL query. The only objective partly met is deployment, which was deliberately limited to a local installation.

7 Discussion

7.1 Effectiveness of techniques and tools

SQL for search and ranking. The specification’s top- N , bottom- N , and group-and-rank searches are a natural fit for window functions. Computing ranks in the database rather than in TypeScript means that pagination, the total count, and the CSV export derive from one ranked subquery and cannot disagree, and that only one page of rows ever leaves the database. The percentile thresholds of the description search benefit similarly: a single `ORDER BY ... LIMIT 1 OFFSET  $k$` query is simpler and faster than loading the values.

Python for reading files. The requirement to read uploads with Python proved sensible: the `csv` module and `openpyxl` handle encodings, byte-order marks, quoted fields, Excel dates, and typed cells with little code. Running the parser as a subprocess with a JSON interface kept the boundary simple: Python decides what a valid row is, and TypeScript decides what to do with it.

One write path in TypeScript. The most valuable structural decision was to give retrieval, upload, and the command-line sync a single write path. Because diffing, upserting, deriving rates, auditing, and issue logging are implemented once in `ingest.ts`, the database-versus-API comparison worked for uploads as soon as it was added, and any fix to the audit rules applies to every source. Static types made such refactoring safe, since changing a record type surfaced every affected call site at compile time.

Validation at every boundary. Every value entering the server from outside, whether a procedure input, a CAMPD row, an environment variable, or language-model output, is parsed with a Zod schema before use, so malformed input fails at the boundary with a readable message rather than deep inside a query.

7.2 Technical choices and alternatives

Next.js, tRPC, and Drizzle instead of Flask, Jinja, and SQLAlchemy. The course suggests Flask with Jinja templates and SQLAlchemy; the team’s proposal specified a Next.js stack instead. The motivation was interactivity: the Explore page updates results as filters change, keeps its state in the URL, and opens details in a dialog over any page, all of which would require a separate JavaScript layer on top of server-rendered Jinja pages. With one language and tRPC, the server’s procedure signatures are the client’s API, so a renamed filter or changed result field becomes a compile error rather than a runtime bug. The cost is a second language for file parsing and a subprocess boundary. Each element of the suggested stack has a direct counterpart (route handlers and procedures for Flask routes, React components for templates, Drizzle for SQLAlchemy, and the same SQLite database), so the objectives concerning routes, templates, forms, and an object-relational mapper are met with equivalent tools.

Drizzle instead of Prisma. Prisma’s query API describes results as nested objects and can express a window function or correlated subquery only through raw SQL strings, which forgo type checking. Drizzle composes queries from typed SQL fragments, so the ranking expression, the per-facility subqueries, and the provenance conditions are ordinary, type-checked code; it also generates readable SQL migrations from the TypeScript schema.

A rule-based parser instead of a language model. A language model would handle more varied phrasing with less code. The rule-based parser was nevertheless chosen as the primary method because it is deterministic, requires no network or key, answers in milliseconds, can be unit-tested exactly, and can only produce values that exist in the database. Its limitation is coverage, since unanticipated phrasing is reported rather than understood. The optional fallback combines the strengths of both: the model sees only the leftover words and may only add vocabulary values, extending coverage without making the core behaviour unpredictable.

Preview-then-save instead of direct synchronisation. The first version of retrieval wrote immediately. The preview costs a second request to the EPA API on save, about two seconds for a full year, but in return the user sees exactly what will change before anything is written, and an interrupted fetch can no longer leave a partial dataset. Where provenance matters, this is the better default; the command-line sync still writes directly for scripted refreshes.

Separate pages with one detail dialog. An earlier version realised the specification’s six pages as tabs and dialogs within a single page. They are now separate routes, so each page has its own address, appears in the browser history, and can be shared; Explore, Map, and Download read their filters from the URL, so a search moves between them intact. Facility, unit, and comparison details remain a dialog rather than a page, because a dialog is closed in one step and returns the user to the exact table row they came from; the open detail is nevertheless written to the URL (`?facility=`, `?unit=`), so it too is addressable.

Storing annual data, fetching sub-annual data. Storing hourly data for every unit would require tens of millions of rows per year. Since Phase 1 searches operate on annual records, only

those are stored; the granular views fetch hourly to monthly data on demand and cache it in the browser.

7.3 Challenges and lessons learned

Combined attribute values. CAMPD’s master-data endpoints list single values such as “Coal” or “Wet Limestone”, but unit records often contain combinations such as “Coal, Pipeline Natural Gas” or “Wet Lime FGD|Wet Limestone”, so equality filters missed many units. Combined values are therefore split into distinct dropdown options, and fuel, type, and control filters match by case-insensitive substring. For the same reason, the “Already in the database” panel can only approximate what an API request with a fuel or control filter will return, which the page states.

Superseded datasets. Because records are keyed by facility, unit, and year, re-retrieving a year moves its records to the new dataset and leaves the old one owning nothing. Deleting the old dataset would erase history, while counting it as active would double the coverage figures. The adopted rule labels an API dataset *superseded* when it once stored records but now owns none; only active datasets are counted, and superseded ones remain visible in the retrieval history.

Generated SQL. The first implementation of that rule returned wrong results. Drizzle omits table prefixes in single-table queries, so in `SELECT ... FROM datasets WHERE NOT EXISTS (SELECT 1 FROM annual_records WHERE dataset_id = id)` the unqualified `id` bound to `annual_records.id` rather than `datasets.id`. Qualifying these subqueries by hand fixed the defect. The lesson is to inspect the SQL an ORM generates whenever a query relates two tables in a non-trivial way.

Ambiguity in natural language. Several two-letter state codes are also common English words (“in”, “or”, “me”, “ok”); treating them as states would turn “units in Texas” into a search for both Indiana and Texas. The parser therefore accepts these codes only in capitals, while full state names and unambiguous codes work in any case. A similar ambiguity exists in the data: the status “Operating (Retired 04/15/2016)” contains the word “Retired”, so the status filter uses a prefix rather than a substring match.

Extent of EPA’s data. CAMPD publishes the current year quarterly, and requests beyond the last published quarter fail. No endpoint reports this date, but the API’s error message names it (“... the quarter ending on 06/30/2026”). The client therefore probes with a full-year request, parses the date from the error, caches it for an hour, and uses it to bound the date pickers of the granular view; if EPA publishes a new quarter while the application runs, a failing request reveals the new date and is retried once.

Missing is not zero. The first schema declared every metric `NOT NULL DEFAULT 0`, which kept sums simple but silently turned a value CAMPD had left blank into a measured zero. The zero-emissions rule then reported 6,105 errors that were in fact reporting gaps, and the heat-rate rule flagged 51 records whose “extreme” rate came from a missing heat input. Making the columns nullable, storing blanks as `NULL`, and adding a separate warning for an unreported CO₂ removed every one of these false errors after a re-sync of all twelve years. The lesson is that absence must be modelled explicitly whenever later logic distinguishes “none” from “unknown”.

Measuring performance. The year-filtered ranking defect (Section 6.4) went unnoticed because every query was fast without a year filter. Indexing each foreign key is not sufficient: indexes should match the combinations of columns that queries filter on together, and the planner’s choice should be verified rather than assumed.

General lessons. Three practices paid off repeatedly. Keeping logic free of database and network access made it testable with fast unit tests. Maintaining one code path per concern (one write path, one CSV writer, one ranked query per view) meant that fixes applied everywhere at once. Finally, never discarding data, by retaining rejected rows, flagged records, and superseded datasets, made every count in the interface explainable.

8 Limitations and future improvements

8.1 Current limitations

Fuel and unit type are not versioned by year. Controls and program codes are stored per unit-year, but fuel and unit type are stored once per unit, and the latest import’s values prevail. They change rarely, but when they do, history is lost. Big Sandy unit BSU1, for example, reported coal as its primary fuel in 2014 but natural gas today, so epaData shows it as a gas unit in every year and a search for coal units in 2014 would miss it.

Records are overwritten, not versioned. Each facility-unit-year has one stored value, and re-importing it overwrites the metrics and reassigns the record to the new dataset. The dataset’s counts record *that* a change occurred and the preview shows *what* changed, but after saving the previous value is lost.

Origin is known per unit-year only. Facility and unit rows have no link to the dataset that created or last modified them, so the Origin filter applies only to annual records.

Local, single-user deployment. The application runs on one machine against one SQLite file and has no user accounts, so anyone who can open it can upload files and run retrievals. An early attempt at serverless hosting required copying the database to temporary storage on every cold start, which loses writes, and was abandoned.

Description search has fixed coverage. The parser does not express negation (“not retired”), alternatives (“coal or gas”), or comparisons between entities (“units larger than Gavin”); such phrases are reported as not understood. Facility names are recognised only after cue words such as “owned by” or “named”.

Upload formats. Uploads recognise the CAMPD annual-emissions and facility-attribute layouts. Hourly and daily files are read, with each row identified by its unit and period, but only their facility and unit attributes are stored, because the schema has no place for sub-annual metrics.

Finally, map clustering and year-over-year spike detection from the proposal were not built (Table 2).

8.2 Future improvements

Phase 2: evaluation. The next phase adds the remaining tables of the specification: TRACI characterisation factors, indicator definitions, calculated indicators, weight scenarios with their weights, and score results. The existing design supports this directly. Indicators are functions of the stored annual records (for example, SO₂ mass multiplied by its acidification factor), so they can be computed with the same ranked queries, exported through the same CSV path, and linked to the dataset of the records from which they were computed.

Data model. Fuel and unit type can be moved onto the annual record in the same way as the controls, since the annual API rows already contain them. A history table receiving every overwritten annual record, together with the dataset that replaced it, would make changes auditable after saving.

Data quality. Year-over-year anomaly detection, flagging a unit whose emissions change by more than a set number of standard deviations of its own history, would complement the physical rules, and a review state on audit flags (open, accepted, or explained) would let analysts record their conclusions.

Search. Negation and alternatives can be added to the filter model as exclusion and multi-value filters, onto which the parser can then map “not” and “or”. Named saved searches would complement the shareable URLs.

Deployment and access. Hosting the database on a libSQL server, which the client library already supports, would permit a shared deployment. This would also require user accounts with upload and retrieval restricted to authorised users, and background jobs for multi-year retrievals.

9 Individual contributions

The Phase 1 proposal divided the work into a back-end and a front-end role, with integration, testing, sample data, and documentation shared. The responsibilities and completed work of each member, as recorded in the project’s version history, are as follows.

Raaj Banga (back-end lead). Responsible for the database schema and indexing, the CAMPD API integration, data validation and anomaly detection, and the search and pagination API. Completed work comprises the schema, its seven migrations, and the index design, including the (`facility_id`, `year`) index added after profiling; the CAMPD client, covering annual retrieval, the preview and comparison with the database, validation and issue logging for API rows, master-data options, the granular time series, and detection of EPA’s published-through quarter; the shared write path (diff, upsert, derived rates, physical-sanity audits) and the provenance model (dataset counts, superseded datasets, record origin); the search back end, comprising the filter, range, and ranking queries, URL state, description search with its percentile thresholds and optional language-model fallback, and the CSV export service; the Explore page, the map and globe, the detail and comparison dialog, the Retrieve and Download pages, and the shared UI components; and the unit tests, the sample upload files, the README, `DATABASE_BREAKDOWN.md`, the demonstration script, and this report.

Arnav Bawankule (front-end lead). Responsible for the user-interface layout, filter forms, map and comparison views, the upload page, and export controls. Completed work comprises the CSV and Excel import feature: the Python parser that maps file columns to the schema and validates every row, the data-quality report of rejected and duplicate rows, the upload endpoint, and the upload page with its preview and approval step, together with its first tests.

The presentation and live demonstration are shared: Arnav presents the interface, upload, detail, and comparison steps, and Raaj the retrieval, search, ranking, and download steps.

10 References and acknowledgments

10.1 Data

All emissions, operating, facility, and unit data come from the U.S. Environmental Protection Agency’s Clean Air Markets Program Data, retrieved through the CAM API (<https://www.epa.gov/power-sector/cam-api-portal>) between September and October 2026. The data are a U.S. government work in the public domain. The rows of the sample upload file are CAMPD data for Kentucky (reporting years 2014 and 2023), retrieved on 9 October 2026, with five rows deliberately altered for testing as documented in `samples/README.md`.

10.2 Reused code and assets

The project was scaffolded with `create-t3-app` (<https://create.t3.gg>), which provided the initial configuration of Next.js, tRPC, Drizzle, Tailwind CSS, and environment-variable validation; none of its example code remains apart from that configuration. The U.S. state and world outlines used by the globe come from the `us-atlas` and `world-atlas` TopoJSON packages by Mike Bostock (ISC licence; <https://github.com/topojson/us-atlas>), bundled in `public/-geo/`. Map tiles are provided by CARTO and OpenStreetMap contributors. All other code depends on the open-source libraries listed in Section 4 and `package.json`, used under their respective licences.

10.3 Generative AI disclosure

In accordance with the course policy, we disclose that generative AI was used throughout this project. We used Anthropic’s Claude, through the Claude Code tool, to help write, refactor, and review application code and tests; to investigate defects; to draft and revise the README, the database documentation, the demonstration script, and this report; and to capture the screenshots. The proposal disclosed its own use of AI in the same way.

The team defined the requirements and design decisions, reviewed all generated code and text, and accepted changes only after verifying them with the type checker, linter, and unit tests, with the acceptance queries against hand-written SQL (Section 6), and through manual use of the application. Every number in this report was produced by running the application or a SQL query against the committed database. The team takes responsibility for the content of the code and of this report.

Bibliography

- [1] Jane Bare. TRACI 2.0: The tool for the reduction and assessment of chemical and other environmental impacts 2.0. *Clean Technologies and Environmental Policy*, 13(5):687–696,

2011.

- [2] Michael Bostock, Vadim Ogievetsky, and Jeffrey Heer. D³: Data-driven documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, 2011.
- [3] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory — ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer, 2001.
- [4] Edgar F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [5] Fred J. Damerau. A technique for computer detection and correction of spelling errors. *Communications of the ACM*, 7(3):171–176, 1964.
- [6] Kevin P. Gaffney, Martin Prammer, Larry Brasfield, D. Richard Hipp, Dan Kennedy, and Jignesh M. Patel. SQLite: Past, present, and future. *Proceedings of the VLDB Endowment*, 15(12):3535–3547, 2022.
- [7] Rob J. Hyndman and Yanan Fan. Sample quantiles in statistical packages. *The American Statistician*, 50(4):361–365, 1996.
- [8] Viktor Leis, Kan Kundhikanjana, Alfons Kemper, and Thomas Neumann. Efficient processing of window functions in analytical SQL queries. *Proceedings of the VLDB Endowment*, 8(10):1058–1069, 2015.

We used AI to help with this project.